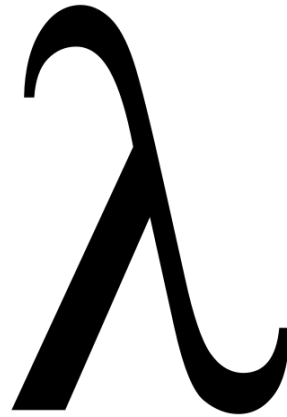


Llenguatges de Programació

lambda càlcul



Albert Rubio, Jordi Petit, Fernando Orejas, Gerard Escudero

UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



Contingut

- **Introducció**
- Estructura bàsica
- Codificacions de Church
- Recursivitat
- Universalitat
- Calculadores

Introducció

El λ -càlcul és un model de computació funcional, l'origen dels llenguatges funcionals, i la base de la seva implementació.

Inventat per Alonzo Church, cap al 1930.



AN UNSOLVABLE PROBLEM OF ELEMENTARY NUMBER THEORY.¹

By ALONZO CHURCH.

1. **Introduction.** There is a class of problems of elementary number theory which can be stated in the form that it is required to find an effectively calculable function f of n positive integers, such that $f(x_1, x_2, \dots, x_n) = 2^k$ is a necessary and sufficient condition for the truth of a certain proposition of elementary number theory involving $x_1, x_2, \dots, x_n$ as free variables.

An example of such a problem is the problem to find a means of determining of any given positive integer n whether or not there exist positive integers x, y, z , such that $x^2 + y^2 = z^2$. For this may be interpreted, required to find an effectively calculable function f , such that $f(n)$ is equal to 2 if and only if there exist positive integers x, y, z , such that $x^2 + y^2 = z^2$. Clearly the condition that the function f be effectively calculable is an essential part of the problem, since without it the problem becomes trivial.

Another example of a problem of this class is, for instance, the problem of topology, to find a complete set of effectively calculable invariants of closed three-dimensional simplicial manifolds under homeomorphisms. This problem can be interpreted as a problem of elementary number theory in view of the fact that topological complexes are representable by matrices of incidence. In fact, as is well known, the property of a set of incidence matrices that it represent a closed three-dimensional manifold, and the property of two sets of incidence matrices that they represent homeomorphic complexes, can both be described in purely number-theoretic terms. If we enumerate, in a straightforward way, the sets of incidence matrices which represent closed three-dimensional manifolds, it will then be immediately provable that the problem under consideration (to find a complete set of effectively calculable invariants of closed three-dimensional manifolds) is equivalent to the problem, to find an effectively calculable function f of positive integers, such that $f(m, n)$ is equal to 2 if and only if the m -th set of incidence matrices and the n -th set of incidence matrices in the enumeration represent homeomorphic complexes. Other examples will readily occur to the reader.

¹ Presented to the American Mathematical Society, April 19, 1935.

² The selection of the particular positive integer 2 instead of some other is, of course, accidental and non-essential.

Consisteix en agafar una línia de símbols i aplicar una operació de *cut-and-paste*.

$(\lambda \textcircled{y}. x(\textcircled{y}z)) (ab)$

↓

$x(abz)$

Fotos: Fair Use, [jstor.org](https://www.jstor.org), [Lambda Calculus for Absolute Dummies](https://www.lambda-calculus.com)

Contingut

- Introducció
- Estructura bàsica
 - Components
 - Avaluació
 - Macros
- Codificacions de Church
- Recursivitat
- Universalitat
- Calculadores

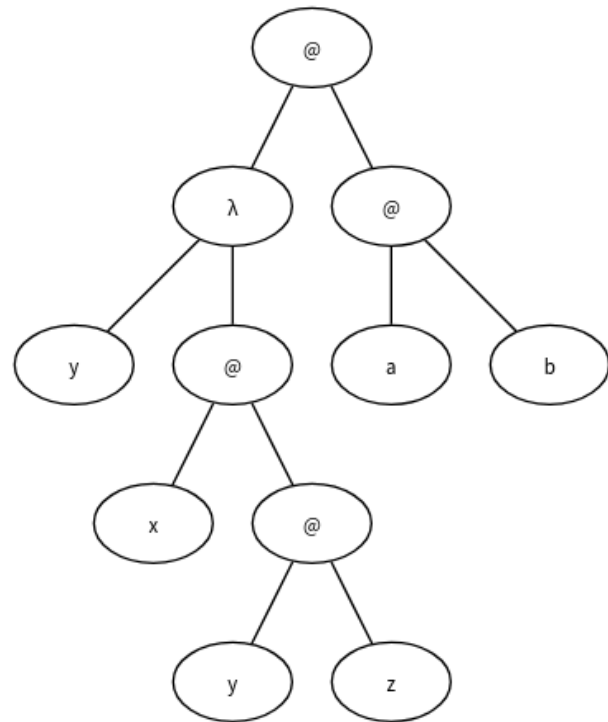
Gramàtica

```
terme    → lletra | ( terme ) | abstracció | aplicació
abstracció → λ lletra . terme
aplicació  → terme terme
```

Exemples de termes:

- $\$x\$$
- $\$\lambda x . x\$$
- $\$(\lambda y . x(yz)) (ab)\$$

Arbre de $\$(\lambda y . x(yz)) (ab)\$$:



Gramàtica

Les lletres es diuen *variables* i no tenen cap significat. El seu nom no importa. Si dues variables tenen el mateix nom, són la mateixa cosa.

Els parèntesis agrupen termes. Per claredat, s'agrupen per l'esquerra:

$\$ \$ a b c d \backslash \text{equiv } (((a b) c) d).$ $\$ \$$

La λ amb el punt introdueix funcions. Per claredat, es poden agrupar λ s: $\$ \$ \lambda x . \lambda y . a \backslash \text{equiv } \lambda x . (\lambda y . a) \backslash \text{equiv } \lambda xy. a$ $\$ \$$

Operacions

Només hi ha dues operacions per la construcció de termes:

- L'**abstracció** captura la idea de definir una funció amb un paràmetre: $\lambda x. u$ on u és un terme.

Diem que λx és el *cap* i que u és el *cos*.

Intuïció: $f(x, y) = x^2 + 2y + x - 1$ és representat per $\lambda x. \lambda y. x^2 + 2y + x - 1$

- L'**aplicació** captura la idea d'aplicar una funció sobre un paràmetre: $f\ x$ on f i x són dos termes.

Intuïció: $f(x)$ és representat per $f\ x$

Currficació

Al λ -càlcul totes les funcions tenen un sol paràmetre.

Les funcions que normalment consideràriem que tenen més d'un paràmetre es representen com a funcions d'un sol paràmetre utilitzant la tècnica del *currying*:

- Una funció amb dos paràmetres, com ara la suma, $+: \text{int} \times \text{int} \rightarrow \text{int}$, es pot considerar equivalent a una funció d'un sol paràmetre que retorna una funció d'un paràmetre, $+: \text{int} \rightarrow (\text{int} \rightarrow \text{int})$.
- Això vol dir que $2 + 3$, amb notació prefixa $(+ 2 \sim 3)$, s'interpretaria com $(+2) \sim 3$, on $(+2)$ és la funció que aplicada a qualsevol paràmetre x , retorna $x+2$.

Currficar és transformar una funció que accepta n paràmetres i convertir-la en una funció que, donat un paràmetre (el primer) retorna una funció que accepta $n-1$ paràmetres (i són semànticament equivalents).

Contingut

- Introducció
- Estructura bàsica
 - Components
 - Avaluació
 - Macros
- Codificacions de Church
- Recursivitat
- Universalitat
- Calculadores

Computació

La **β -reducció** (*cut-and-paste*) és la regla essencial de computació del λ -càlcul:

$$(\lambda x . u) \ v \longrightarrow_{\beta} u[x:=v]$$

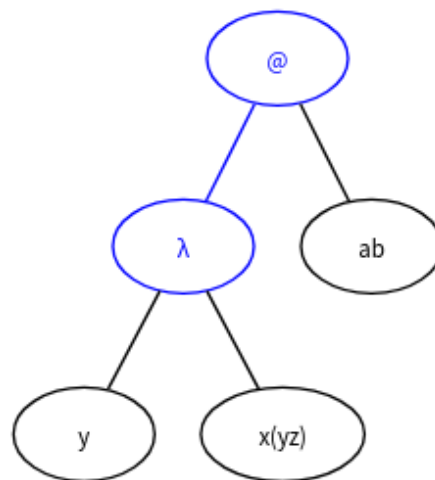
on $u[x:=v]$ vol dir reescriure u substituint les seves x per v .

Exemple:

expressió	acció efectuada
$(\lambda y.x(yz))(ab)$	β -reducció de y
$x((ab)z)$	$=$
$x(abz)$	

No cal que aparegui el patró a l'arrel de l'arbre.

Patró de la β -reducció.



Forma normal

Si una expressió no pot β -reduir-se, aleshores es diu que està en **forma normal**.

Si $t \rightarrow \dots \rightarrow t'$ i t' està en forma normal, aleshores es diu que t' és la forma normal de t , i es considera que t' és el resultat de l'avaluació de t .

Una λ -expressió té, com a màxim, una forma normal.

Variables lliures i lligades

Dins d'un terme, una variable és **lligada** si apareix al cap d'una funció que la conté. Altrament és **lliure**.

Les variables poden ser lliures i lligades alhora en un mateix terme.

Per exemple: $$(\lambda x.xy)(\lambda y.y)$$

- y és lliure a la primera subexpressió.
- y és lligada a la segona subexpressió.

El problema de la captura de noms I

Quan s'aplica la β -reducció s'ha de tenir cura amb els noms de les variables i, si cal, reanomenar-les.

El problema es pot veure en el següent exemple: Sigui $\text{TWICE} \equiv \lambda f. \lambda x. f(f\ x)$:

Calculem $\text{TWICE} \sim \text{TWICE}$:

expressió	acció efectuada
$\text{TWICE} \ \text{TWICE}$	definició de TWICE
$(\lambda f. \lambda x. f(fx)) \ \text{TWICE}$	β -reducció de f
$(\lambda x. \text{TWICE}(\text{TWICE} \ x)) \ \text{TWICE}$	definició de TWICE
$(\lambda x. \text{TWICE}(\lambda f. \lambda x. f(f\ x)) \ x) \ \text{TWICE}$	

El problema de la captura de noms II

Aplicant la β -reducció directament tindríem:

expressió	acció efectuada
$(\lambda x.\text{TWICE}(\lambda f.\lambda x.f(f\ x))\ x)\ \backslash\ \backslash\ \$$	β -reducció de $f\$$
$(\lambda x.\text{TWICE}(\lambda x.x(x\ x)))\$$	ERROR

El que hauríem de fer és reanomenar la variable lligada x més interna:

expressió	acció efectuada
$(\lambda x.\text{TWICE}(\lambda f.\lambda x.f(f\ x))\ x)\ \backslash\ \backslash\ \$$	canvi de nom $x \rightarrow y$
$(\lambda x.\text{TWICE}((\lambda f.\lambda y.f(f\ y))\ x))\$$	β -reducció de $f\$$
$(\lambda x.\text{TWICE}((\lambda y.x(x\ y)))\$$	OK

α -Conversió

A més de la β -reducció, al λ -càlcul tenim la regla de l' α -conversió per reanomenar les variables. Per exemple:

$$\lambda x . \lambda y . xy \rightarrow_a \lambda z . \lambda y . zy \rightarrow_a \lambda z . \lambda t . zt$$

Aleshores l'exemple del `\text{TWICE}` el podríem escriure:

expressió	acció efectuada
<code>\text{TWICE} ~ \text{TWICE} \$</code>	definició de <code>\text{TWICE} \$</code>
<code>(\lambda f. \lambda x. f(f x)) \text{TWICE} \$</code>	β -reducció de <code>f \$</code>
<code>(\lambda x. \text{TWICE} (\text{TWICE} ~ x)) \$</code>	definició de <code>\text{TWICE} \$</code>
<code>(\lambda x. \text{TWICE} (\lambda f. \lambda x. f(f x)) \ x) \$</code>	α -conversió <code>[x/y]</code>
<code>(\lambda x. \text{TWICE} ((\lambda f. \lambda y. f(f y)) \ x)) \ \ \$</code>	β -reducció de <code>f \$</code>
<code>(\lambda x. \text{TWICE} ((\lambda y. x(x y)) \$</code>	

Ordres de reducció

Donada una λ -expressió, pot haver més d'un lloc on es pot aplicar β -reducció, per exemple:

\$\$ (1) \sim (\lambda x . x((\lambda z . zz)x)) t \longrightarrow t((\lambda z . zz)t) \longrightarrow t(tt) \$\$

però també:

\$\$ (2) \sim (\lambda x . x((\lambda z . zz)x)) t \longrightarrow (\lambda x . x(xx)) t \longrightarrow t(tt) \$\$

Hi ha dues formes estàndard d'avaluar una λ -expressió:

- Avaluació en **ordre normal**: s'aplica l'estratègia **left-most outer-most**:
Reduir la λ sintàcticament més a l'esquerra (1).
- Avaluació en **ordre aplicatiu**: s'aplica l'estratègia **left-most inner-most**:
Reduir la λ més a l'esquerra de les que són més endins (2).

Ordres de reducció

En principi, podríem pensar que no importa l'ordre d'avaluació que utilitzem, perquè la β -reducció és **confluent**:

Si $t \rightarrow \dots \rightarrow t_1$ i $t \rightarrow \dots \rightarrow t_2$ llavors
 $t_1 \rightarrow \dots \rightarrow t_3$ i $t_2 \rightarrow \dots \rightarrow t_3$

Tanmateix, si una expressió té una forma normal, aleshores la reducció en ordre normal la trobarà, però no necessàriament la reducció en ordre aplicatiu.

Per exemple, en ordre normal tenim:

$((\lambda x.a) ((\lambda y.yy) (\lambda z.zz))) \rightarrow a$

però en ordre aplicatiu:

$((\lambda x.a) ((\lambda y.yy) (\lambda z.zz))) \rightarrow (\lambda x.a) ((\lambda z.zz) (\lambda z.zz)) \rightarrow \dots$

Contingut

- Introducció
- Estructura bàsica
 - Components
 - Avaluació
 - Macros
- Codificacions de Church
- Recursivitat
- Universalitat
- Calculadores

Macros

En el λ -càlcul, les funcions no reben noms.

Per facilitar-ne la escriptura, utilitzarem **macros** que representen funcions i les expandirem quan calgui, com vam fer a les transparències anteriors amb $\text{\texttt{\textit{TWICE}}}$.

Les macros també es diuen **combinadors**.

⇒ És un recurs "meta" que no forma part del llenguatge (preprocessador).

Exemple: $\text{\texttt{\textit{ID}}} \equiv \lambda x.x$

Llavors:

expressió	acció efectuada
$\text{\texttt{\textit{ID}}} \ \text{\texttt{\textit{ID}}}$	definició $\text{\texttt{\textit{ID}}}$
$(\lambda x.x) \ \text{\texttt{\textit{ID}}} \ \text{\texttt{\textit{ID}}} \ \text{\texttt{\textit{ID}}}$	β -reducció de x
$\text{\texttt{\textit{ID}}}$	

Contingut

- Introducció
- Estructura bàsica
- Codificacions de Church
 - Booleans
 - Naturals
 - Enters
- Recursivitat
- Universalitat
- Calculadores

Codificació de Church

Com es representen els tipus dades i els seus operadors en λ -càlcul.

Naturals en λ -càlcul: Una codificació estranya?

Dec	Bin	Romà	Xinès	Devanagari	λ -càlcul
0	0		零	०	$\lambda sz.z$
1	1	I	一	१	$\lambda sz.sz$
2	10	II	二	२	$\lambda sz.s(sz)$
3	11	III	三	३	$\lambda sz.s(s(sz))$
4	100	IV	四	४	$\lambda sz.s(s(s(sz)))$
$\vdots$				$\vdots$	

El natural n és l'aplicació d' n cops la funció s a z .

L'important no és com es representen els naturals, sinó establir una bijecció entre la seva representació i $\mathbb{N}$.

Tampoc estem considerant-ne l'eficiència.

Contingut

- Introducció
- Estructura bàsica
- Codificacions de Church
 - Booleans
 - Naturals
 - Enters
- Recursivitat
- Universalitat
- Calculadores

Booleans I

Church encoding*:

- $T \equiv \lambda t. \lambda f. t$ el primer
- $F \equiv \lambda t. \lambda f. f$ el segon

Com fem el not ?

- $\text{not} \equiv \lambda g. gFT$ "flip"
- $\text{not} \ T \rightarrow (\lambda g. gFT)T \rightarrow TFT \rightarrow \dots \rightarrow F$ el primer
- $\text{not} \ F \rightarrow (\lambda g. gFT)F \rightarrow FFT \rightarrow \dots \rightarrow T$ el segon

Exercici: completar les β -reduccions.

* Church encoding - Wikipedia

Booleans II

Com fem el condicional?

- $\text{if} \equiv \lambda c. \lambda x. \lambda y. cxy$
- **Exercicis:** codificar i avaluar:
 - `if F then poma else pera`
 - `if T then poma else pera`

el 1er o el 2on?

Com fem l'\$and\$?

- `and x y = if x then y else F`
- $\text{and} \equiv \lambda x. \lambda y. xyF$
- **Exercici:** demostreu l'anterior.

I l'\$or\$?

- ...

Contingut

- Introducció
- Estructura bàsica
- Codificacions de Church
 - Booleans
 - Naturals
 - Enters
- Recursivitat
- Universalitat
- Calculadores

Funcions aritmètiques bàsiques

- $0 \equiv \lambda s. \lambda z. z \equiv F$
- $n \equiv \lambda s. \lambda z. s^n z$

Com fem el succ ?

- $\text{succ} = \lambda n. \lambda f. \lambda x. f(nfx)$ 1a f per afegir, fx per consumir
- **Exercici:** avalueu
 - $\text{succ } 1$

Com fem la suma ?

- $\text{suma} \equiv \lambda m. \lambda n. n \text{ succ } m$ succ per cada f de la n
- **Exercici:** avalueu aplica n vegades succ a m
 - $\text{suma } 2 \text{ } 1$

Més funcions aritmètiques

Altres:

- $\text{mul} \equiv \lambda m. \lambda n. \lambda f. n(mf)$
- $\text{power} \equiv \lambda m. \lambda n. nm$
- **Exercici:**

penseu el perquè, interpreteu-les

Avançats:

- $\text{minus} \equiv \lambda m. \lambda n. n \text{ pred } m$
- $\text{pred} \equiv \lambda n. \lambda f. \lambda x. n(\lambda g. \lambda h. h(gf))(\lambda u. x)(\lambda u. u)$

Predicats

isZero?

- $\text{isZero} \equiv \lambda n. n(\lambda x. F)T$ 0 consumeix F i és queda la T
- **Exercici:** avalueu n es queda F i $\lambda x. F$ descarta la resta
 - $\text{isZero } 0$
 - $\text{isZero } 2$

Relacionals:

- $\text{leq} \equiv \lambda m. \lambda n. \text{IsZero } (\text{minus } m \ n)$
- $\text{seq} \equiv \lambda m. \lambda n. \text{and } (\text{leq } m \ n) (\text{leq } n \ m)$

Contingut

- Introducció
- Estructura bàsica
- Codificacions de Church
 - Booleans
 - Naturals
 - Enters
- Recursivitat
- Universalitat
- Calculadores

Tuples

Parells:

- $\text{pair} \equiv \lambda x. \lambda y. \lambda p. pxy$
- Exemple: $\text{pair} \ 2 \ 3 \equiv \lambda p. p \ 2 \ 3$

Accés:

- $\text{first} \equiv \lambda p. p(\lambda x. \lambda y. x)$
- $\text{second} \equiv \lambda p. p(\lambda x. \lambda y. y)$

Exercici: avalueu

- $\text{first} \ (\text{pair} \ 2 \ 3)$
- $\text{second} \ (\text{pair} \ 2 \ 3)$

Enters

Els codifiquem amb una resta en un parell:

```
2 = pair 2 0  
-3 = pair 0 3
```

Funcions:

- $\$convert \equiv \lambda x. pair\ x\ 0$ natural a enter
- $\$neg \equiv \lambda x. pair\ (second\ x)\ (first\ x)$

S'utilitza una funció `$oneZero` per generar parells amb almenys un zero (amb recursivitat, `$Y$`).

Totes les funcions aritmètiques es generen tenint el compte els parells.

De la mateixa forma els racionals són parells d'enters.

Les llistes també es codifiquen a partir de parells (com en Lisp).

Contingut

- Introducció
- Estructura bàsica
- Codificacions de Church
- Recursivitat
- Universalitat
- Calculadores

Rekursivitat

Combinador Y (paradoxal o de punt fix):

$$Y \equiv \lambda y. (\lambda x. y(xx)) (\lambda x. y(xx))$$

Compleix la propietat:

$$YR \equiv R(YR)$$

Demostració:

expressió	acció efectuada
$Y \ \backslash \ R$	definició de Y
$(\lambda y. (\lambda x. y(xx)) (\lambda x. y(xx))) \backslash \backslash \backslash \ R$	β -reducció de y
$(\lambda x. R(xx)) (\lambda x. R(xx))$	β -reducció de x
$R((\lambda x. R(xx)) (\lambda x. R(xx)))$	per aquest resultat i l'anterior
$R(YR)$	

Factorial I

El combinador Y ens permet definir la funció factorial. Sigui:

$$H \equiv \lambda f. \lambda n. \text{IF} (n=0) \sim 1 \sim (n \times (f \sim (n-1)))$$

podem veure com YH funciona com el factorial:

expressió	acció efectuada
$Y H 1$	combinador Y
$H(Y H) 1$	definició de H
$(\lambda f. \lambda n. \text{IF} (n=0) 1 (n \times (f (n-1)))) (Y H) 1$	β -reducció de f
$(\lambda n. \text{IF} (n=0) 1 (n \times (Y H (n-1)))) 1$	β -reducció de n
$\text{IF} (1=0) 1 (1 \times (Y H (1-1)))$	$\text{IF} = \text{fals}$
$1 \times (Y H (1-1))$	trivial
$Y H 0$	combinador Y
...	

Factorial II

expressió	acció efectuada
$\$Y\ H\ 0\$$	combinador $\$Y\$$
$\$H\ (Y\ H)\ 0\$$	definició de $\$H\$$
$\$ \lambda f. \lambda n. \text{IF} (n=0)\ 1\ (n \times (f\ (n-1)))\ (Y\ H)\ 0\ \$$	β -reducció de $\$f\$$
$\$ \lambda n. \text{IF} (n=0)\ 1\ (n \times (Y\ H\ (n-1)))\ 0\$$	β -reducció de $\$n\$$
$\$ \text{IF} (0=0)\ 1\ (0 \times (Y\ H\ (0-1)))\$$	$\$IF=cert\$$
$\$1\$$	

Contingut

- Introducció
- Estructura bàsica
- Codificacions de Church
- Recursivitat
- Universalitat
- Calculadores

Universalitat del λ -càlcul

A partir d'aquí, ja només queda anar continuant fent definicions i anar-les combinant.

Eventualment, es pot arribar a veure que qualsevol algorisme és implementable en λ -càlcul perquè pot simular a una màquina de Turing.

Teorema [Kleene i Rosser, 1936]: Totes les funcions recursives poden ser representades en λ -càlcul ($\iff$ Turing complet).

A diferència de les màquines de Turing que són un model matemàtic d'una màquina *hardware* imperativa, el λ -càlcul només utilitza reescriptura i és un model matemàtic més *software* i funcional.

λ -càlcul amb tipus: existeixen extensions amb tipus; que són les que solen utilitzar els llenguatges funcionals com model.

Contingut

- Introducció
- Estructura bàsica
- Codificacions de Church
- Recursivitat
- Universalitat
- Calculadores

Calculadores

Existeixen moltes calculadores de λ -càlcul *online*:

- https://www.cl.cam.ac.uk/~rmk35/lambda_calculus/lambda_calculus.html
- <https://jacksongl.github.io/files/demo/lambda/index.htm>
- <http://www-cs-students.stanford.edu/~blynn/lambda/> (amb notació Haskell)